

TP de chiffrement à clé secrète

Copyright © 2018 Wenliang Du.

Cette œuvre est mise à disposition selon les termes de la licence Creative Commons Attribution – Pas d'Utilisation Commerciale – Partage dans les Mêmes Conditions 4.0 International.

1 Vue d'ensemble

L'objectif pédagogique de ce TP est de familiariser les étudiants avec les concepts du chiffrement à clé secrète et avec certaines attaques courantes contre le chiffrement. Grâce à ce TP, les étudiants acquerront une expérience pratique des algorithmes de chiffrement, des modes de chiffrement, du bourrage (padding) et du vecteur d'initialisation (IV). De plus, les étudiants seront capables d'utiliser des outils et d'écrire des programmes pour chiffrer/déchiffrer des messages.

De nombreuses erreurs courantes ont été commises par les développeurs dans l'utilisation des algorithmes et des modes de chiffrement. Ces erreurs affaiblissent la robustesse du chiffrement et conduisent finalement à des vulnérabilités. Ce TP expose les étudiants à certaines de ces erreurs et leur demande de lancer des attaques pour exploiter ces vulnérabilités. Ce TP couvre les sujets suivants :

- Chiffrement à clé secrète
- Chiffrement par substitution et analyse fréquentielle
- Modes de chiffrement, IV et bourrage (padding)
- Erreurs courantes dans l'utilisation des algorithmes de chiffrement
- Programmation avec la bibliothèque de cryptographie

Lectures. Une couverture détaillée du chiffrement à clé secrète se trouve dans :

- Le chapitre 21 du livre SEED, *Computer & Internet Security: A Hands-on Approach*, 3^e édition, par Wenliang Du

Environnement du TP. Ce TP a été testé sur la VM Ubuntu préconfigurée SEED, téléchargeable depuis le site web du projet SEED. Il peut aussi être réalisé sur toute machine Linux disposant des outils `openssl`, `python3`, `gcc` et de la bibliothèque `libssl-dev`.

2 Environnement du TP

Dans ce TP, nous utilisons un conteneur pour exécuter un oracle de chiffrement. Le conteneur n'est nécessaire que pour la tâche 6.3 ; il n'est donc pas nécessaire de le démarrer pour les autres tâches.

Installation du conteneur et commandes. Veuillez télécharger le fichier `Labsetup.zip` depuis le site web du TP, le décompresser, entrer dans le dossier `Labsetup`, et utiliser le fichier `docker-compose.yml` pour mettre en place l'environnement du TP.

```
$ docker-compose build    # Construire les images des conteneurs
$ docker-compose up       # Démarrer les conteneurs
$ docker-compose down     # Arrêter les conteneurs

// Alias couramment utilis (définis dans .bashrc sur la VM SEED)
$ dcbuild                 # alias pour : docker-compose build
$ dcup                    # alias pour : docker-compose up
$ dcdown                   # alias pour : docker-compose down
```

Tous les conteneurs s'exécutent en arrière-plan. Pour obtenir un shell sur un conteneur, utilisez d'abord `docker ps` pour trouver son ID, puis `docker exec` :

```
$ dockps          // alias pour : docker ps --format "{{.ID}}  {{.Names}}"
$ docksh <id>     // alias pour : docker exec -it <id> /bin/bash

// Exemple : obtenir un shell dans hostC
$ dockps
b1004832e275  hostA-10.9.0.5
0af4ea7a3e2e  hostB-10.9.0.6
9652715c8e0a  hostC-10.9.0.7

$ docksh 96
root@9652715c8e0a:/#
```

3 Tâche 1 : Analyse fréquentielle

Il est bien connu que le chiffrement par substitution monoalphabétique (aussi appelé chiffre monoalphabétique) n'est pas sûr, car il est vulnérable à l'analyse fréquentielle. Dans ce TP, on vous donne un texte chiffré à l'aide d'un chiffre monoalphabétique ; c'est-à-dire que chaque lettre du texte original est remplacée par une autre lettre, et ce remplacement ne varie pas (une lettre est toujours remplacée par la même lettre pendant le chiffrement). Votre travail consiste à retrouver le texte original par analyse fréquentielle. On sait que le texte original est un article en anglais.

Ci-dessous, nous décrivons comment nous avons chiffré l'article original et quelles simplifications nous avons faites. Les enseignants peuvent utiliser la même méthode pour chiffrer un article de leur choix, au lieu de demander aux étudiants d'utiliser le texte chiffré que nous avons produit.

- **Étape 1** : générons la clé de chiffrement, c'est-à-dire la table de substitution. Nous allons permuter l'alphabet de a à z avec Python et utiliser l'alphabet permuté comme clé. Voir le programme suivant.

```
#!/bin/env python3

import random
s = "abcdefghijklmnopqrstuvwxyz"
list = random.sample(s, len(s))
key = ''.join(list)
print(key)
```

- **Étape 2** : simplifions un peu l'article original. Nous convertissons toutes les majuscules en minuscules, puis nous supprimons toute la ponctuation et les chiffres. Nous conservons les espaces entre les mots, afin que vous puissiez toujours voir les frontières des mots dans le texte chiffré. Dans un vrai chiffrement monoalphabétique, les espaces seraient supprimés. Nous gardons les espaces pour simplifier la tâche. Nous avons fait cela avec les commandes suivantes :

```
$ tr [:upper:] [:lower:] < article.txt > lowercase.txt
$ tr -cd '[a-z][\n][:space:]' < lowercase.txt > plaintext.txt
```

- **Étape 3** : nous utilisons la commande `tr` pour effectuer le chiffrement. Nous ne chiffons que les lettres, en laissant les espaces et les retours à la ligne inchangés.

```
$ tr 'abcdefghijklmnopqrstuvwxyz' 'sxtrwinqbedpvgkfmalhyuojzc' \
  < plaintext.txt > ciphertext.txt
```

Nous avons créé un texte chiffré avec une clé de chiffrement différente (pas celle décrite ci-dessus). Il est inclus dans le fichier `Labsetup.zip`, téléchargeable depuis le site web du TP. Votre travail consiste à utiliser l'analyse fréquentielle pour retrouver la clé de chiffrement et le texte en clair original.

Nous avons également fourni un programme Python (`freq.py`) dans le dossier `Labsetup/Files`. Il lit le fichier `ciphertext.txt` et produit des statistiques sur les n-grammes, y compris les fréquences des lettres seules, des bigrammes (séquences de 2 lettres) et des trigrammes (séquences de 3 lettres), etc.

```
$ ./freq.py
-----
1-gram (top 20):
n: 488
y: 373
v: 348
...
-----
2-gram (top 20):
yt: 115
tn: 89
mu: 74
...
-----
3-gram (top 20):
ytn: 78
vup: 30
mur: 20
...
```

Conseils. Grâce à l'analyse fréquentielle, vous pouvez retrouver assez facilement le texte en clair de certains caractères. Pour ces caractères, vous pouvez les remplacer par leur équivalent en clair, car cela peut vous donner davantage d'indices. Il est préférable d'utiliser des majuscules pour le texte en clair, afin que, pour une même lettre, on sache laquelle est en clair et laquelle est chiffrée. Vous pouvez utiliser la commande `tr` pour cela. Par exemple, dans la commande suivante, nous remplaçons les lettres `a`, `e` et `t` dans `in.txt` par les lettres `X`, `G` et `E`, respectivement ; les résultats sont enregistrés dans `out.txt`.

```
$ tr 'aet' 'XGE' < in.txt > out.txt
```

De nombreuses ressources en ligne peuvent vous aider. Nous en listons quelques-unes utiles ci-dessous :

- https://en.wikipedia.org/wiki/Frequency_analysis : cette page Wikipédia fournit les fréquences d'un texte anglais typique.
- <https://en.wikipedia.org/wiki/Bigram> : fréquences des bigrammes.
- <https://en.wikipedia.org/wiki/Trigram> : fréquences des trigrammes.

4 Tâche 2 : Chiffrement avec différents algorithmes et modes

Dans cette tâche, nous allons expérimenter divers algorithmes et modes de chiffrement. Vous pouvez utiliser la commande `openssl enc` suivante pour chiffrer/déchiffrer un fichier. Pour consulter les manuels, vous pouvez taper `man openssl` et `man enc`.

```
$ openssl enc -ciphertext -e -in plain.txt -out cipher.bin \  
-K 00112233445566778889aabbccddeeff \  
-iv 0102030405060708
```

Veillez remplacer `ciphertext` par un type de chiffrement précis, tel que `-aes-128-cbc`, `-bf-cbc`, `-aes-128-cfb`, etc. Dans cette tâche, vous devez essayer au moins 3 algorithmes différents. Vous pouvez trouver la signification des options de la ligne de commande et tous les types de chiffrement pris en charge en tapant "`man enc`". Nous listons ci-dessous quelques options courantes de la commande `openssl enc` :

<code>-in <file></code>	fichier d'entree
<code>-out <file></code>	fichier de sortie
<code>-e</code>	chiffrer
<code>-d</code>	dechiffrer
<code>-K/-iv</code>	cle/IV en hexadecimal (argument suivant)
<code>-[pP]</code>	afficher l'IV/la cle (puis quitter si -P)

5 Tâche 3 : Mode de chiffrement – ECB contre CBC

Le fichier `pic_original.bmp` est inclus dans le fichier `Labsetup.zip` ; c'est une image simple. Nous voulons chiffrer cette image afin que les personnes ne possédant pas les clés de chiffrement ne puissent pas savoir ce qu'elle contient. Veillez chiffrer le fichier en utilisant les modes ECB (Electronic Code Book) et CBC (Cipher Block Chaining), puis effectuez les étapes suivantes :

1. Traitons l'image chiffrée comme une image et utilisons un logiciel de visualisation d'images pour l'afficher. Cependant, pour un fichier `.bmp`, les 54 premiers octets contiennent les informations d'en-tête de l'image ; nous devons les définir correctement afin que le fichier chiffré puisse être traité comme un fichier `.bmp` valide. Nous allons remplacer l'en-tête de l'image chiffrée par celui de l'image originale. Nous pouvons utiliser l'éditeur hexadécimal `bless` (déjà installé sur notre VM) pour modifier directement des fichiers binaires. Nous pouvons aussi utiliser les commandes suivantes pour récupérer l'en-tête de `p1.bmp`, les données de `p2.bmp` (de l'offset 55 jusqu'à la fin du fichier), puis combiner l'en-tête et les données dans un nouveau fichier.

```
$ head -c 54 p1.bmp > header  
$ tail -c +55 p2.bmp > body  
$ cat header body > new.bmp
```

2. Affichez l'image chiffrée à l'aide d'un programme de visualisation d'images (nous avons installé sur notre VM un visualiseur appelé `eog`). Pouvez-vous déduire des informations utiles sur l'image originale à partir de l'image chiffrée ? Veillez expliquer vos observations.

Choisissez une image de votre choix, répétez l'expérience ci-dessus et rapportez vos observations.

6 Tâche 4 : Bourrage (Padding)

Pour les chiffrements par blocs, lorsque la taille d'un texte en clair n'est pas un multiple de la taille du bloc, un bourrage peut être nécessaire. Le schéma de bourrage PKCS#5 est largement utilisé par de nombreux chiffrements par blocs (voir le chapitre 21.4 du livre SEED pour plus de détails). Nous allons mener les expériences suivantes pour comprendre le fonctionnement de ce type de bourrage :

1. Utilisez les modes ECB, CBC, CFB et OFB pour chiffrer un fichier (vous pouvez choisir n'importe quel algorithme). Veuillez indiquer quels modes utilisent un bourrage et lesquels n'en utilisent pas. Pour ceux qui n'ont pas besoin de bourrage, veuillez expliquer pourquoi.
2. Créons trois fichiers contenant respectivement 5 octets, 10 octets et 16 octets. Nous pouvons utiliser la commande `"echo -n"` suivante pour créer de tels fichiers. L'exemple suivant crée un fichier `f1.txt` de longueur 5 (sans l'option `-n`, la longueur serait de 6, car un caractère de nouvelle ligne serait ajouté par `echo`) :

```
$ echo -n "12345" > f1.txt
```

Nous utilisons ensuite `"openssl enc -aes-128-cbc -e"` pour chiffrer ces trois fichiers avec AES 128 bits en mode CBC. Veuillez décrire la taille des fichiers chiffrés.

Nous aimerions voir ce qui est ajouté comme bourrage pendant le chiffrement. Pour y parvenir, nous allons déchiffrer ces fichiers avec `"openssl enc -aes-128-cbc -d"`. Malheureusement, par défaut, le déchiffrement supprime automatiquement le bourrage, ce qui rend impossible de le voir. Cependant, la commande dispose d'une option appelée `"-nopad"`, qui désactive le bourrage, c'est-à-dire que, pendant le déchiffrement, la commande ne supprimera pas les données de bourrage. Par conséquent, en regardant les données déchiffrées, nous pouvons voir quelles données ont été utilisées pour le bourrage. Veuillez utiliser cette technique pour déterminer quels bourrages sont ajoutés aux trois fichiers.

Notez que les données de bourrage peuvent ne pas être imprimables ; vous devez donc utiliser un outil hexadécimal pour afficher le contenu. L'exemple suivant montre comment afficher un fichier au format hexadécimal :

```
$ hexdump -C p1.txt
00000000 31 32 33 34 35 36 37 38 39 49 4a 4b 4c 0a  |123456789IJKL.|
$ xxd p1.txt
00000000: 3132 3334 3536 3738 3949 4a4b 4c0a          123456789IJKL.
```

7 Tâche 5 : Propagation d'erreurs – Texte chiffré corrompu

Pour comprendre la propriété de propagation d'erreurs des différents modes de chiffrement, nous voulons réaliser l'exercice suivant :

1. Créez un fichier texte d'au moins 1000 octets.
2. Chiffrez le fichier avec le chiffrement AES-128.
3. Malheureusement, un seul bit du 55^e octet du fichier chiffré a été corrompu. Vous pouvez provoquer cette corruption avec l'éditeur hexadécimal **bleess**.
4. Déchiffrez le fichier chiffré corrompu avec la bonne clé et le bon IV.

Veuillez répondre à la question suivante : quelle quantité d'information pouvez-vous récupérer en déchiffrant le fichier corrompu, si le mode de chiffrement est ECB, CBC, CFB ou OFB, respectivement ? Veuillez répondre à cette question avant de réaliser cette tâche, puis vérifiez si votre réponse est correcte ou fausse une fois la tâche terminée. Veuillez fournir une justification.

8 Tâche 6 : Vecteur d'initialisation (IV) et erreurs courantes

La plupart des modes de chiffrement nécessitent un vecteur d'initialisation (IV). Les propriétés d'un IV dépendent du schéma cryptographique utilisé. Si nous ne faisons pas attention au choix

des IV, les données que nous chiffons peuvent ne pas être sûres du tout, même si nous utilisons un algorithme et un mode de chiffrement sûrs. L'objectif de cette tâche est d'aider les étudiants à comprendre les problèmes qui surviennent lorsqu'un IV n'est pas choisi correctement. Les directives détaillées pour cette tâche sont fournies dans le chapitre 21.5 du livre SEED.

8.1 Tâche 6.1. Expérience sur l'IV

Une exigence de base pour l'IV est *l'unicité*, ce qui signifie qu'aucun IV ne doit être réutilisé avec la même clé. Pour comprendre pourquoi, veuillez chiffrer le même texte en clair en utilisant (1) deux IV différents, et (2) le même IV. Veuillez décrire votre observation et, à partir de celle-ci, expliquer pourquoi l'IV doit être unique.

8.2 Tâche 6.2. Erreur courante : utiliser le même IV

On pourrait soutenir que, si le texte en clair ne se répète pas, utiliser le même IV est sans danger. Examinons le mode Output Feedback (OFB). Supposons que l'attaquant obtienne un texte en clair (P1) et un texte chiffré (C1) ; peut-il déchiffrer d'autres messages chiffrés si l'IV est toujours le même ? On vous donne les informations suivantes ; essayez de retrouver le contenu réel de P2 à partir de C2, P1 et C1.

Texte en clair	(P1): This is a known message!
Texte chiffré	(C1): a469b1c502c1cab966965e50425438e1bb1b5f9037a4c159
Texte en clair	(P2): (inconnu de vous)
Texte chiffré	(C2): bf73bcd3509299d566c35b5d450337e1bb175f903fafc159

Si nous remplaçons OFB dans cette expérience par CFB (Cipher Feedback), quelle part de P2 peut être révélée ? Vous devez seulement répondre à la question ; il n'est pas nécessaire de le démontrer.

L'attaque utilisée dans cette expérience est appelée *attaque à texte clair connu* (known-plaintext attack), un modèle d'attaque en cryptanalyse dans lequel l'attaquant a accès à la fois au texte en clair et à sa version chiffrée (texte chiffré). Si cela peut conduire à la révélation d'autres informations secrètes, le schéma de chiffrement n'est pas considéré comme sûr.

Exemple de code. Nous fournissons un programme d'exemple appelé `sample_code.py`, que vous trouverez dans le dossier `Labsetup/Files`. Il montre comment effectuer un XOR de chaînes (chaînes ASCII et chaînes hexadécimales). Le code est le suivant :

```
#!/usr/bin/python3

# XOR de deux tableaux d'octets
def xor(first, second):
    return bytearray(x^y for x,y in zip(first, second))

MSG = "A message"
HEX_1 = "aabbccddeeff1122334455"
HEX_2 = "1122334455778800aabbdd"

# Conversion chaine ascii/hex en tableau d'octets
D1 = bytes(MSG, 'utf-8')
D2 = bytearray.fromhex(HEX_1)
D3 = bytearray.fromhex(HEX_2)

r1 = xor(D1, D2)
r2 = xor(D2, D3)
r3 = xor(D2, D2)
```

```
print(r1.hex())
print(r2.hex())
print(r3.hex())
```

8.3 Tâche 6.3. Erreur courante : utiliser un IV prévisible

D'après les tâches précédentes, nous savons maintenant que les IV ne peuvent pas se répéter. Une autre exigence importante sur l'IV est que les IV doivent être imprévisibles pour de nombreux schémas, c'est-à-dire qu'ils doivent être générés aléatoirement. Dans cette tâche, nous allons voir ce qui se passe si les IV sont prévisibles.

Supposons que Bob vienne d'envoyer un message chiffré, et qu'Ève sache que son contenu est soit **Yes**, soit **No** ; Ève peut voir le texte chiffré et l'IV utilisé pour chiffrer le message, mais comme l'algorithme de chiffrement AES est très robuste, Ève n'a aucune idée du contenu réel. Cependant, comme Bob utilise des IV prévisibles, Ève sait exactement quel IV Bob va utiliser ensuite.

Un bon chiffrement ne doit pas seulement résister à l'attaque à texte clair connu décrite précédemment, il doit aussi résister à *l'attaque à texte clair choisi* (chosen-plaintext attack), un modèle d'attaque en cryptanalyse dans lequel l'attaquant peut obtenir le texte chiffré de n'importe quel texte en clair. Comme AES est un chiffrement robuste qui résiste à l'attaque à texte clair choisi, Bob ne voit aucun inconvénient à chiffrer n'importe quel texte en clair fourni par Ève ; il utilise bien un IV différent pour chaque texte en clair, mais malheureusement, les IV qu'il génère ne sont pas aléatoires et peuvent toujours être prédits.

Votre travail consiste à construire un message et à demander à Bob de le chiffrer et de vous donner le texte chiffré. Votre objectif est d'utiliser cette occasion pour déterminer si le contenu réel du message secret de Bob est **Yes** ou **No**. Pour cette tâche, on vous fournit un oracle de chiffrement qui simule Bob et chiffre les messages avec AES 128 bits en mode CBC. Vous pouvez accéder à l'oracle en exécutant la commande suivante :

```
$ nc 10.9.0.80 3000
Bob's secret message is either "Yes" or "No", without quotations.
Bob's ciphertex: 54601f27c6605da997865f62765117ce
The IV used      : d27d724f59a84d9b61c0f2883efa7bbc

Next IV         : d34c739f59a84d9b61c0f2883efa7bbc
Your plaintext  : 11223344aabbccdd
Your ciphertex: 05291d3169b2921f08fe34449ddc3611

Next IV         : cd9f1ee659a84d9b61c0f2883efa7bbc
Your plaintext  : <your input>
```

Après vous avoir montré le prochain IV, l'oracle vous demandera de saisir un message en clair (sous forme de chaîne hexadécimale). L'oracle chiffrera le message avec le prochain IV et affichera le nouveau texte chiffré. Vous pouvez essayer différents textes en clair, mais gardez à l'esprit qu'à chaque fois, l'IV change, mais reste prévisible. Pour vous simplifier la tâche, nous avons fait en sorte que l'oracle affiche le prochain IV. Pour quitter l'interaction, appuyez sur **Ctrl+C**.

8.4 Lectures complémentaires

Il existe des cryptanalyses plus avancées sur l'IV qui dépassent le cadre de ce TP. Les étudiants peuvent lire l'article publié à cette adresse : <https://defuse.ca/cbcmodeiv.htm>. Comme les exigences sur l'IV dépendent réellement du schéma cryptographique, il est difficile de se souvenir des propriétés à respecter lors du choix d'un IV. Cependant, nous serons en sécurité si nous utilisons toujours un nouvel IV pour chaque chiffrement, et si ce nouvel IV est généré à l'aide d'un

bon générateur de nombres pseudo-aléatoires, afin qu'il soit imprévisible pour les adversaires. Consultez un autre TP SEED (Random Number Generation Lab) pour plus de détails sur la façon de générer des nombres pseudo-aléatoires cryptographiquement forts.

9 Tâche 7 : Programmation avec la bibliothèque de cryptographie

Cette tâche est principalement conçue pour les étudiants en informatique/ingénierie ou dans des domaines connexes, où la programmation est requise. Les étudiants doivent vérifier auprès de leurs professeurs si cette tâche est obligatoire pour leurs cours ou non.

Dans cette tâche, on vous donne un texte en clair et un texte chiffré, et votre travail consiste à trouver la clé utilisée pour le chiffrement. Vous connaissez les faits suivants :

- Le chiffrement **aes-128-cbc** est utilisé.
- La clé utilisée pour chiffrer ce texte en clair est un mot anglais de moins de 16 caractères ; le mot peut être trouvé dans un dictionnaire anglais courant. Comme le mot compte moins de 16 caractères (soit 128 bits), des signes dièse (**#** : valeur hexadécimale **0x23**) sont ajoutés à la fin du mot pour former une clé de 128 bits.

Votre objectif est d'écrire un programme pour retrouver la clé de chiffrement. Vous pouvez télécharger une liste de mots anglais sur Internet. Nous en avons aussi inclus une dans le fichier **Labsetup.zip**. Le texte en clair, le texte chiffré et l'IV sont indiqués ci-dessous :

```
Texte en clair (21 caracteres) : This is a top secret.  
Texte chiffre (hexadecimal): 764aa26b55a4da654df6b19e4bce00f4  
                             ed05e09346fb0e762583cb7da2ac93a2  
IV (hexadecimal):          aabbccddeeff00998877665544332211
```

Vous devez faire attention aux points suivants :

- Si vous choisissez de stocker le message en clair dans un fichier et de fournir ce fichier à votre programme, vous devez vérifier que la longueur du fichier est de 21. Si vous saisissez le message dans un éditeur de texte, sachez que certains éditeurs peuvent ajouter un caractère spécial à la fin du fichier. Le moyen le plus simple de stocker le message dans un fichier est d'utiliser la commande suivante (l'option **-n** indique à **echo** de ne pas ajouter de retour à la ligne final) :

```
$ echo -n "This is a top secret." > file
```

- Dans cette tâche, vous devez écrire votre propre programme qui appelle la bibliothèque de cryptographie. Aucun point ne sera accordé si vous utilisez simplement les commandes **openssl** pour réaliser cette tâche. Un exemple de code est disponible à l'adresse suivante :

```
https://www.openssl.org/docs/man1.1.1/man3/EVP\_CipherInit.html
```

- Lorsque vous compilez votre code avec **gcc**, n'oubliez pas d'inclure l'option **-lcrypto**, car votre code a besoin de la bibliothèque **crypto**. Voir l'exemple suivant :

```
$ gcc -o myenc myenc.c -lcrypto
```

Note pour les enseignants. Nous encourageons les enseignants à générer leur propre texte en clair et texte chiffré avec une clé différente ; ainsi, les étudiants ne pourront pas obtenir la réponse ailleurs ou à partir de cours précédents. Les enseignants peuvent utiliser les commandes suivantes pour y parvenir (veuillez remplacer le mot **exemple** par un autre mot secret, et ajouter le bon nombre de signes # pour que la longueur de la chaîne soit de 16) :

```
$ echo -n "This is a top secret." > plaintext.txt
$ echo -n "exemple#####" > key
$ xxd -p key
6578616d706c65232323232323232323
$ openssl enc -aes-128-cbc -e -in plaintext.txt -out ciphertext.bin \
  -K 6578616d706c652323232323232323 \
  -iv 010203040506070809000a0b0c0d0e0f \
$ xxd -p ciphertext.bin
e5accdb667e8e569b1b34f423508c15422631198454e104ceb658f5918800c22
```

10 Rendu

Vous devez rendre un rapport de TP détaillé, avec des captures d'écran, décrivant ce que vous avez fait et ce que vous avez observé. Vous devez également fournir une explication des observations intéressantes ou surprenantes. Veuillez aussi lister les extraits de code importants, suivis d'une explication. Le simple fait de joindre du code sans explication ne donnera droit à aucun point.

11 Remerciements

Nous tenons à remercier les personnes et organisations suivantes pour leur contribution :

- Jiamin Shen a développé les éléments suivants : le code s'exécutant dans le conteneur, et la version conteneurisée de la tâche sur l'IV prévisible.
- La National Science Foundation des États-Unis a financé le projet SEED de 2002 à 2020.
- L'Université de Syracuse a fourni les ressources du projet SEED à partir de 2001.